# Job-Market Intelligence Platform

**Designing a System That Monetizes Data — Big Data Systems Final Project**

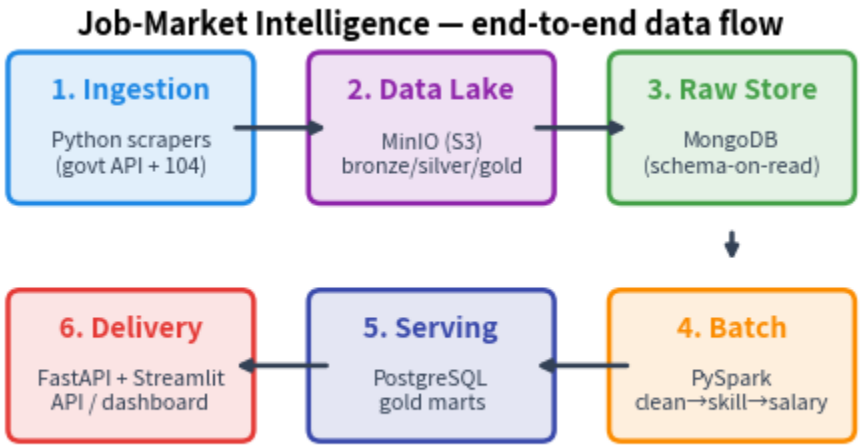
**Student ID:** r14922128 | National Taiwan University | Spring 2026

**GitHub repository:** https://github.com/yande001/r14922128-bda-job-market-intelligence **Live demo:** runs locally via `make demo` (see README); not publicly hosted.

---

## Executive Summary

Raw job postings are abundant and public, but the *decision-useful* form of that data — "which skills are rising, and what do they pay, right now, in Taiwan" — is not sold by anyone in a real-time, skill-level, time-series form. This project builds an end-to-end big-data system that ingests job postings, extracts in-demand skills, normalizes salaries, and serves **skill-demand and salary-benchmark** products through an API and a dashboard. The business case targets coding bootcamps and university career centers first (clear pain, real budget), with a freemium funnel for job seekers and a data feed for recruiters.




---

## 1. Target Customer (Component 1)

"Everyone who looks for a job" is not a customer. We pick a **wedge** and expand outward.

### Primary wedge — coding bootcamps & university career centers (B2B2X)

Examples: ALPHA Camp, AppWorks School, Hahow/course providers, and university career offices (e.g., NTU's career center).

- **Job they are trying to do:** decide *which skills to teach / advise* so their graduates get hired at good salaries, and prove that outcome to prospective students.
- **Status quo:** ad-hoc manual scanning of 104 listings, annual third-party salary PDFs, and gut feel. Stale, coarse, and labor-intensive.
- **Why we are better:** we give them a continuously-updated, skill-level view of what employers demand and pay — the exact input their curriculum and marketing need. They have **budget** (curriculum is their core product) and a **measurable ROI** (placement rate, marketing claims).

### Secondary segments

- **Job seekers / career-switchers (freemium B2C):** "which skill should I learn next, and what salary should I ask for?" Free skill trends; paid personalized gap+salary report. This segment is also our cheapest acquisition channel and a source of first-party survey data.
- **Recruiters / HR / staffing (B2B data feed):** monthly skill-demand and compensation-benchmark feed via the API to calibrate offers and job specs.

### Why this segmentation works

The B2C funnel is cheap to reach and generates demand signal; the B2B and B2B2X segments convert that signal into revenue. The same gold marts power all three — one pipeline, three products.

---

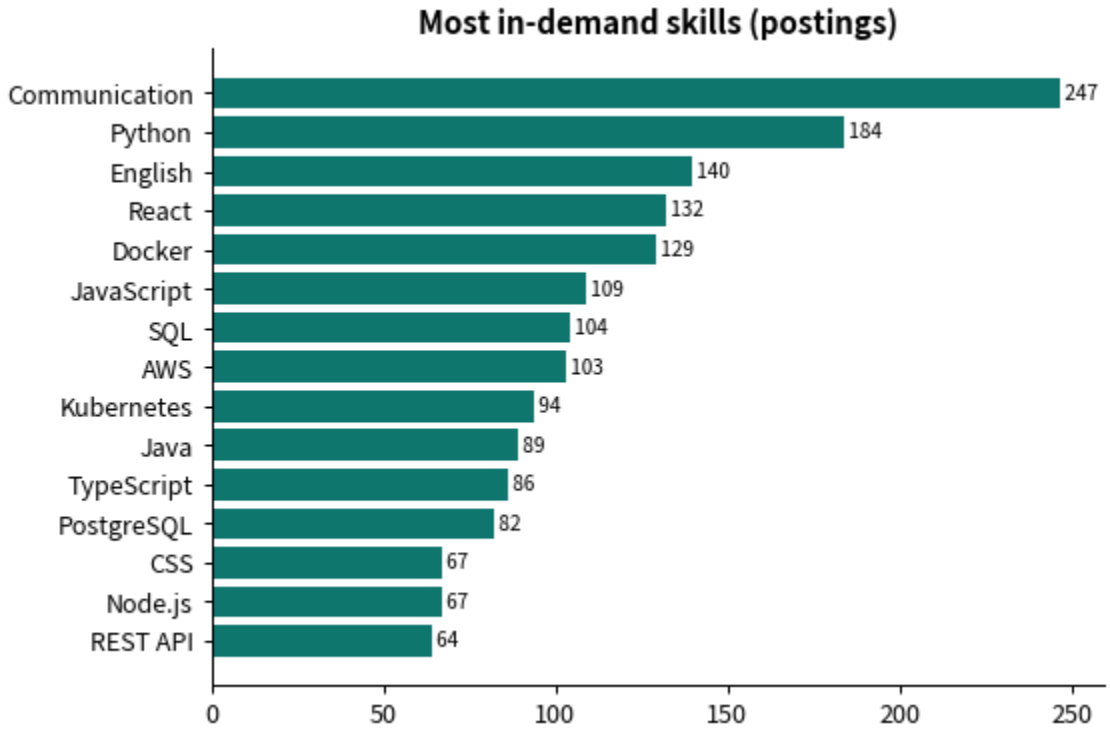
## 2. Evidence of Demand & Willingness to Pay (Component 2)

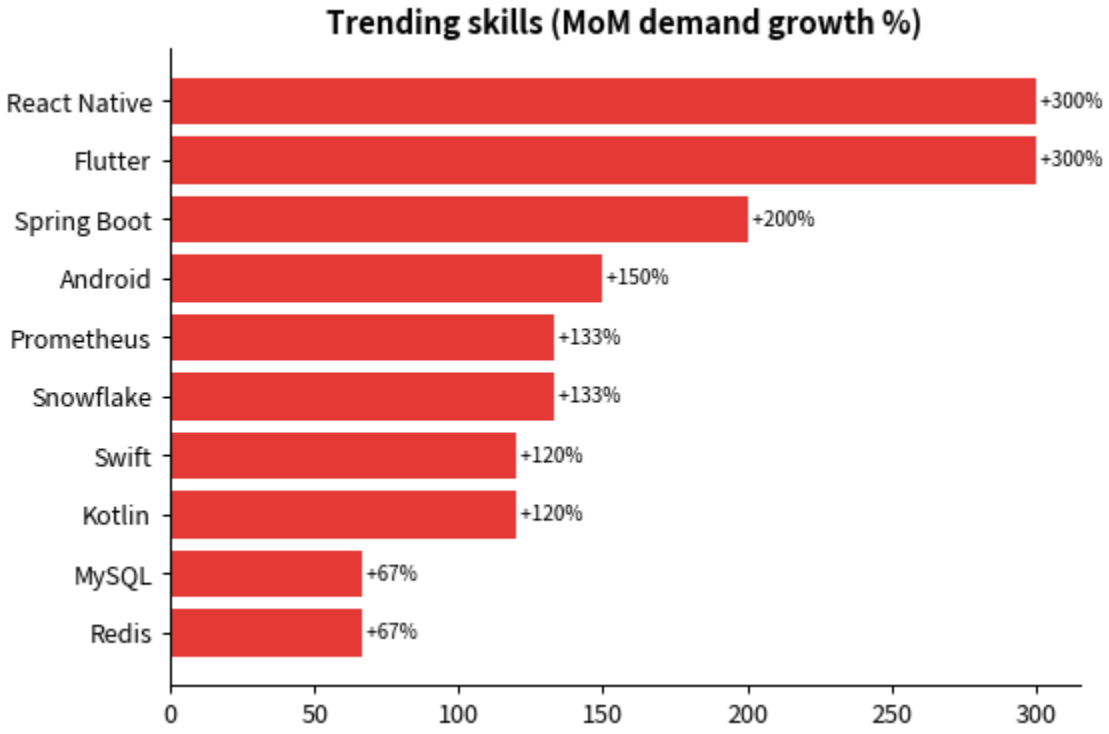
This is the heart of the project: how we went from a hunch to a defensible claim. The data-acquisition

pipeline does double duty — it is both the technical system and the demand evidence. Full reproduction steps are in `notebooks/demand_analysis.ipynb`.

### 2.1 The corpus is the primary evidence

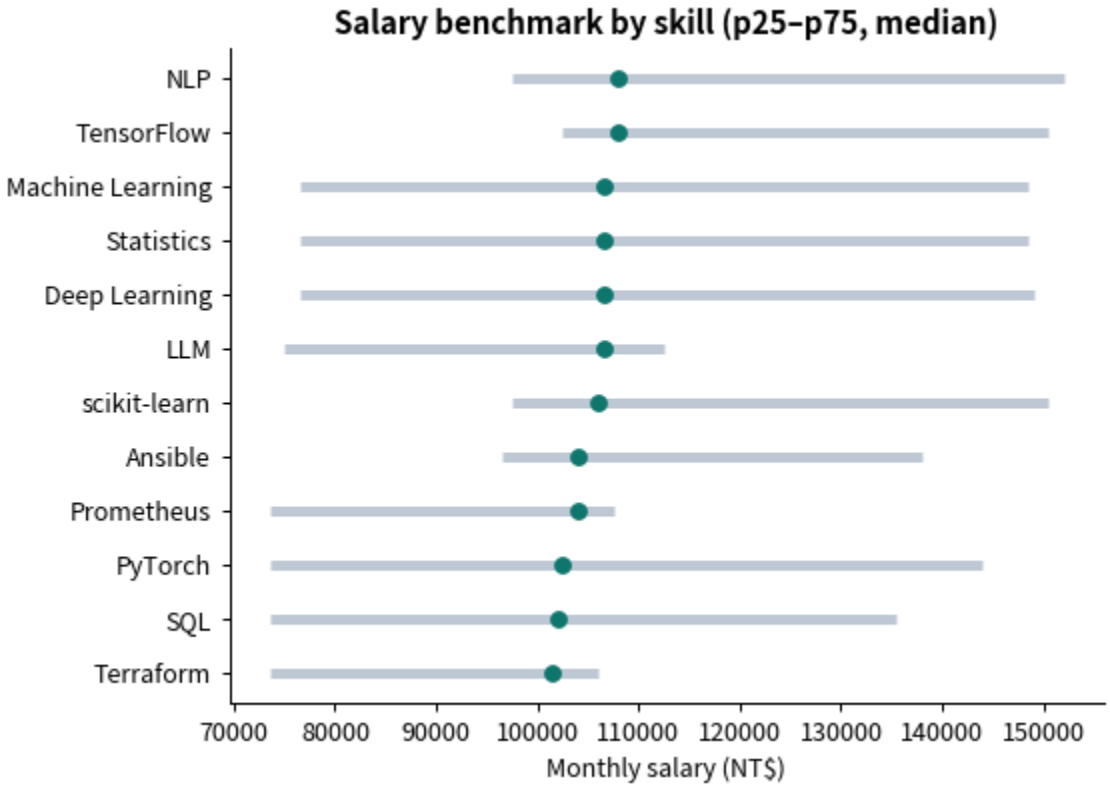
We built the full acquisition pipeline (Section 3) and, even on a sample corpus, recover a ranked, categorized skill-demand table with month-over-month momentum and skill-level salary percentiles. This proves two things at once: **employers demand these skills**, and **the salary/skill signal is reliably extractable** from public text.





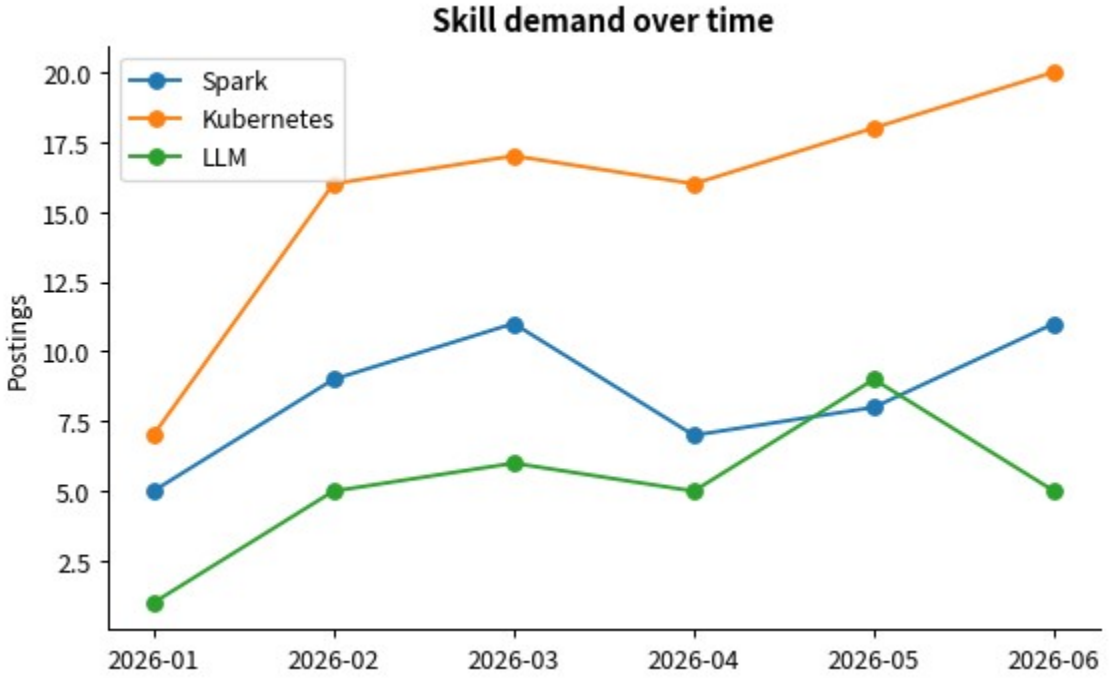

### 2.2 The salary signal competitors don't publish

We compute salary percentiles (p25/p50/p75) per skill, region, and seniority — the granular view incumbents lack for Taiwan.




### 2.3 Demand trends over time

Because each posting carries a date, we produce time series per skill — the basis for "rising skill" alerts and curriculum-timing advice.



### 2.4 Competitor benchmark — a real market with an open cell

| Product | Taiwan-local | Skill-level | Real-time | Time-series |
|---|---|---|---|---|
| Levels.fyi | partial | ✗ | crowd | ✗ |
| Glassdoor / LinkedIn Salary | partial | ✗ | ✗ | ✗ |
| Robert Walters / Michael Page guides | ✓ | ✗ | ✗ (annual) | ✗ |
| 104 薪資情報 / Yourator salary | ✓ | ✗ | ✗ | ✗ |
| **This project** | ✓ | ✓ | ✓ | ✓ |

The existence of paid salary guides and crowdsourced salary sites proves people pay for compensation data. None fills the *real-time × skill-level × Taiwan-local × time-series* cell — our wedge.

### 2.5 Willingness to pay

We propose a 10-minute survey (target n=30–50) anchored on concrete prices: - **Job seekers:** NT$0 / 99 / 299 for a personalized skill-gap + salary report. - **Recruiters:** NT$2k / 5k / 10k per month for a skill-demand + comp feed. - **Bootcamps:** NT$50k / 150k annual data license.

**Non-monetary anchor (time saved):** manually compiling a comparable skill+salary benchmark from 104 takes a recruiter an estimated 6–10 hours/month; at NT$500/hr that is NT$3,000–5,000/month of labor a feed replaces — a credible price floor for the B2B tier.

### 2.6 Public corroboration

Government employment statistics show sustained IT hiring; PTT *Tech_Job* and Dcard threads repeatedly ask "is skill X worth learning?" and "what salary should I ask?" — precisely the questions the product answers, recurring and unmet.

---

## 3. System Design (Technical)

The system is a classic lakehouse pipeline, each stage mapped to a course concept and a concrete tool. Everything runs locally via `docker-compose`.

### 3.1 Data sources & ingestion

- **Primary — 台灣就業通 / Ministry of Labor open data** (data.gov.tw #44062), CSV/JSON/API under the **Government Data Open License v1** (free, commercial reuse allowed). This is the legal, reproducible spine.
- **Enrichment — 104.com.tw** JSON endpoints (`/jobs/search/list`, `/job/ajax/content/{id}`, the latter requiring a `Referer` header) for skill/tool granularity. Used at low volume under the academic exception.

Scrapers (`scrapers/`) implement a polite client (`base.py`): one request at a time per host, configurable delay + jitter, exponential backoff, an honest User-Agent, a hard record cap, and response caching. **No PII is ever stored** — the schema has no recruiter name/email/phone fields, and free text is scrubbed (`scrub_pii`).

### 3.2 Storage

- **MinIO** (S3-compatible) is the **distributed file system / data lake**, organized bronze → silver → gold. Chosen over HDFS because it gives the same `s3a://` object-store semantics Spark consumes, in a single lightweight container.
- **MongoDB** is the **NoSQL raw store**: postings are semi-structured with fields that differ across sources, so a document store fits schema-on-read.

### 3.3 Processing — PySpark batch pipeline

Five jobs (`spark_jobs/`), chained by `run_all.py` in one Spark session: 1. **clean** — bronze JSON → typed, normalized silver (derives `year_month`, `seniority`, `region`, a searchable text blob). 2. **dedup** — collapses the same posting across sources/days via a normalized (company, title, region) key, keeping the latest and recording first/last seen. 3. **salary_norm** — normalizes pay to monthly TWD (year/12, hour×176, day×22), drops implausible values, keeps negotiable as null. 4. **skill_extract** — gazetteer matching

(`scrapers/config/skills.yaml`, ~120 skills with zh/en aliases and categories) over the text blob, exploded to a long posting×skill table. Explainable and fast; skillNer/MLlib is the documented upgrade path. 5. **trend_aggregate** — builds gold marts (skill demand monthly, skill overview with MoM change, skill salary benchmark at three grains, market KPIs) and loads them to PostgreSQL via JDBC.

Spark runs in `local[*]`, appropriate for the data volume; the identical code scales to a cluster by changing the master and pointing `s3a://` at a multi-node store.

### 3.4 Serving & delivery

- **PostgreSQL** holds the gold marts — indexed, low-latency relational serving for point/range/aggregate queries. The Spark loader writes with truncate-overwrite so the schema and indexes (`sql/schema.sql`) survive every rerun.
- **FastAPI** (`api/`) exposes the monetizable feed: `/skills`, `/skills/trending`, `/skills/{skill}/trend`, `/salary`, `/summary`, `/health` — with OpenAPI docs.
- **Streamlit** (`dashboard/`) is the human-facing demo: KPIs, trending skills, demand ranking, salary benchmarks with region/seniority filters, and per-skill time series.

### 3.5 Reproducibility

A committed, deterministic sample corpus (`data/sample_data/`, regenerable via `data/generate_sample.py`) lets the entire pipeline run **offline** — `make demo` brings up the stack, ingests the sample, runs Spark, and populates the dashboard with no network access. Live collection is opt-in (`make scrape`).

### 3.6 Scalability & cost (10× / 100×)

- **10×** (hundreds of thousands of postings): MinIO + a 2–3 node Spark cluster; add a **Kafka** topic so bursty scraping is decoupled from processing; schedule with **Prefect/Airflow**. Commodity-VM cost, low four figures NT$/month.
- **100×** (millions, multi-country): partition the lake by source/region/month, move serving to a managed Postgres or a columnar store, cache hot API responses. Cost scales sub-linearly because aggregates are tiny relative to raw text; the expensive part is batch compute, which is periodic, not always-on.

## 4. Go-to-Market Difficulties (Bonus Component 3)

We take the obstacles seriously — building the pipeline is the easy half.

- **Trust & adoption.** Buyers must trust our numbers over an incumbent's. Mitigation: publish methodology, sample sizes, and coverage %; let customers drill from an aggregate to the (anonymized) underlying counts.
- **Data-acquisition cost & legality.** Our richest source (104) is *not* licensed for commercial resale; `robots.txt` gates bots. The free government feed is fully licensed but broader and less skill-granular. A commercial launch must license 104, strike a data partnership, or lean on the government feed plus first-party data. This is the single biggest risk and is documented in `docs/data_ethics.md`.
- **Legal & privacy (PDPA).** Taiwan's PDPA treats commercial monetization as use beyond the original purpose (Art. 20), with criminal penalties for unlawful profit from personal data (Art. 41). Our mitigation is structural: **store no PII, sell only aggregates** — even the raw layer is PII-free by construction.
- **Cold start.** Demand trends need history. Mitigation: bootstrap from the government feed's existing records, and start collecting immediately so time series accrue; the free B2C tier seeds usage and survey data while history builds.
- **Competition & moats.** Anyone can scrape; the moat is *accumulated time-series*, the cleaned skill gazetteer, customer relationships, and (eventually) licensed data others can't legally resell.
- **Unit economics.** Marginal cost per customer is near zero (the marts are shared); the cost is periodic batch compute. Profitability arrives at a few paying B2B/B2B2X accounts, since one pipeline serves all tiers.

---

## 5. Conclusion

We built a complete, reproducible big-data system that turns free public job-posting data into a real-

time, skill-level salary & demand product — and a business case for who pays and why. The corpus proves the signal exists and is extractable; the competitor landscape shows a market with an unfilled cell; and a time-saved framing gives a defensible price. The technical design uses each course tool where it genuinely fits, and the whole thing runs with a single `make demo`.